

VN-ALPR

Hệ thống nhận diện biển số xe Việt Nam
cho cổng bãi đỗ xe

Báo cáo dự án Deep Learning

Đoàn Trí Hùng

Mã nguồn: <https://github.com/hungdt31/vn-alpr>

Tháng 10 năm 2026

Tóm tắt

Bối cảnh. Báo cáo trình bày VN-ALPR, một hệ thống nhận diện biển số xe Việt Nam từ đầu đến cuối (end-to-end), phục vụ việc ghi nhận xe vào/ra tại cổng bãi đỗ xe. Hệ thống phải đọc được cả biển 1 dòng của ô tô lẫn biển 2 dòng của xe máy, và chạy được thời gian thực trên video.

Phương pháp. Hệ thống gồm năm khối: (1) bộ phát hiện biển số YOLO11n được fine-tune; (2) khối tiền xử lý cắt, chỉnh nghiêng và tách biển 2 dòng bằng hồ sơ chiếu ngang (horizontal projection profile); (3) bộ nhận dạng ký tự CRNN (CNN + BiLSTM + CTC) tự huấn luyện; (4) khối hậu xử lý khớp chuỗi với các mẫu định dạng biển số Việt Nam và sửa ký tự dễ nhầm theo vị trí; (5) khối theo dõi đa đối tượng ByteTrack kết hợp bỏ phiếu có trọng số qua nhiều khung hình. Do thiếu dữ liệu chữ thật đã gắn nhãn, CRNN được huấn luyện trước trên dữ liệu tổng hợp do dự án tự sinh. Hệ thống được đóng gói thành REST API (FastAPI), giao diện web (Streamlit) và cơ sở dữ liệu SQLite ghi lượt vào/ra.

Kết quả. Sau epoch đầu tiên, bộ phát hiện đạt mAP@0.5 bằng 0,969 trên tập kiểm định, vượt mục tiêu 0,95. CRNN đạt độ chính xác dòng 85,4 % trên biển tổng hợp chưa từng thấy. Toàn bộ pipeline chạy khoảng 62 ms/ảnh (khoảng 16 FPS) trên CPU laptop, vượt mục tiêu 8 FPS.

Kết luận. Phân tích lỗi trên ảnh thật cho thấy bộ phát hiện và khối tiền xử lý hoạt động tốt, còn điểm nghẽn chính là khoảng cách miền (domain gap) giữa font tổng hợp và font biển số Việt Nam ở bộ nhận dạng. Fine-tune CRNN trên ảnh cắt từ biển thật là bước cải thiện ưu tiên.

Từ khoá: nhận diện biển số xe, YOLO, CRNN, CTC, ByteTrack, OCR, dữ liệu tổng hợp, biển số Việt Nam.

Mục lục

1	Giới thiệu	4
1.1	Bài toán	4
1.2	Mục tiêu	4
1.3	Đóng góp	4
2	Cơ sở lý thuyết và công trình liên quan	4
3	Dữ liệu	5
3.1	Dữ liệu phát hiện biển số	5
3.2	Dữ liệu nhận dạng chữ tổng hợp	6
3.3	Định dạng nhãn chữ thật	6
4	Phương pháp	6
4.1	Kiến trúc tổng thể	6
4.2	Phát hiện biển số	7
4.3	Tiền xử lý	7
4.4	Nhận dạng ký tự: CRNN	8
4.5	Hậu xử lý theo định dạng biển số Việt Nam	8
4.6	Theo dõi và bỏ phiếu qua nhiều khung hình	9
4.7	Lớp ứng dụng	9
5	Thiết lập thực nghiệm	10
5.1	Môi trường	10
5.2	Siêu tham số	10
5.3	Độ đo	10
6	Kết quả	11
6.1	Bộ phát hiện	11
6.2	Bộ nhận dạng trên dữ liệu tổng hợp	11
6.3	Ablation end-to-end	11
6.4	Tốc độ	12
7	Phân tích lỗi	12
7.1	Nghiên cứu tình huống: biển 54-L1 9999	12
7.2	Nhận xét chung	13
8	Hạn chế và hướng phát triển	13

9	Kết luận	14
A	Cấu trúc mã nguồn	14
B	Tái lập kết quả	15

1 Giới thiệu

1.1 Bài toán

Tại cổng ra vào bãi đỗ xe của chung cư, trung tâm thương mại hay nhà máy, việc ghi biển số thường làm thủ công hoặc dùng thẻ giấy. Cách làm này chậm, dễ sai và khó tra cứu. Bài toán nhận diện biển số tự động (Automatic License Plate Recognition, ALPR) nhận ảnh hoặc video từ camera và trả về chuỗi ký tự của biển số. Kết quả đó được dùng để ghi nhận lượt vào/ra và tính thời gian gửi xe.

Biển số Việt Nam có hai đặc thù làm bài toán khó hơn so với nhiều nước khác:

- **Có cả biển 1 dòng và 2 dòng.** Ô tô thường dùng biển 1 dòng (ví dụ 51G-123.45). Xe máy và một số ô tô dùng biển 2 dòng (ví dụ 59-X1 / 123.45).
- **Định dạng có cấu trúc nhưng đa dạng.** Seri có thể gồm 1–2 chữ cái, hoặc chữ cái kèm chữ số với xe máy, và phần số có 4–5 chữ số.

1.2 Mục tiêu

Các chỉ tiêu được đặt ra trong đặc tả dự án như sau:

Bảng 1: Chỉ tiêu mục tiêu của dự án.

Chỉ số	Mục tiêu	Ghi chú
mAP@0.5 của bộ phát hiện	$\geq 0,95$	Trên tập test
mAP@0.5:0.95 của bộ phát hiện	$\geq 0,70$	
Độ chính xác ký tự	$\geq 97\%$	
Độ chính xác biển (khớp hoàn toàn)	$\geq 90\%$	Chỉ số quan trọng nhất
Tốc độ end-to-end trên GPU	≥ 25 FPS	Video 720p
Tốc độ end-to-end trên CPU	≥ 8 FPS	Laptop không có GPU

1.3 Đóng góp

1. Một pipeline hoàn chỉnh, hỗ trợ cả biển 1 dòng và 2 dòng, từ phát hiện, tách dòng, nhận dạng, hậu xử lý đến theo dõi qua video.
2. Thuật toán hậu xử lý dựa trên mẫu định dạng biển số Việt Nam, sửa ký tự theo vị trí và chọn mẫu cần ít lần sửa nhất.
3. Bộ sinh biển số tổng hợp để huấn luyện trước CRNN khi chưa có dữ liệu chữ thật.
4. Ứng dụng demo gồm REST API, giao diện web và cơ sở dữ liệu vào/ra, kèm bộ kiểm thử tự động chạy được khi chưa có mô hình.

2 Cơ sở lý thuyết và công trình liên quan

Phát hiện đối tượng. Họ mô hình YOLO [1] giải bài toán phát hiện đối tượng trong một lần lan truyền (single-stage), dự đoán đồng thời khung bao và độ tin cậy. Dự án dùng YOLO11 [2],

phiên bản không dùng anchor (anchor-free) của Ultralytics. Bản *nano* chỉ có khoảng 2,6 triệu tham số, phù hợp chạy thời gian thực.

Nhận dạng chuỗi ký tự. CRNN [3] kết hợp CNN để trích đặc trưng ảnh với RNN hai chiều để mô hình hoá chuỗi, và được huấn luyện bằng hàm mất mát CTC [4]. Ưu điểm của CTC là không cần biết vị trí của từng ký tự trong ảnh, chỉ cần chuỗi đáp án của cả dòng. PaddleOCR [6] là một hệ thống OCR tổng quát mạnh, được dùng làm baseline để so sánh.

Theo dõi đa đối tượng. ByteTrack [5] theo hướng *tracking-by-detection*: nó ghép các khung phát hiện giữa các khung hình bằng bộ lọc Kalman và thuật toán ghép cặp, đồng thời tận dụng cả những khung có độ tin cậy thấp. Nhờ vậy mỗi xe giữ được một mã định danh (track ID) ổn định.

Định dạng biển số Việt Nam. Ký hiệu D là chữ số và L là chữ cái. Phần đầu biển (mã tỉnh và seri) thuộc một trong các mẫu ở Bảng 2, theo sau là 4–5 chữ số.

Bảng 2: Các mẫu đầu biển số được hỗ trợ.

Mẫu	Ví dụ	Loại xe	Cách hiển thị
DDL	51G-123.45	Ô tô	1 hoặc 2 dòng
DDLL	30LD-123.45	Ô tô (seri 2 chữ)	1 hoặc 2 dòng
DDLD	59-X1 123.45	Xe máy	2 dòng
DDLLD	29-MĐ1 123.45	Xe máy điện (chỉ seri MĐ)	2 dòng

3 Dữ liệu

3.1 Dữ liệu phát hiện biển số

Dữ liệu ảnh lấy từ bộ *vietnamese-license-plate* (phiên bản 1) trên Roboflow Universe, workspace `school-fuhih`. Nhãn ở định dạng YOLO với một lớp duy nhất là `plate`. Ba tập con có sẵn của Roboflow được gộp lại rồi chia lại theo tỉ lệ 70/15/15 với hạt giống ngẫu nhiên cố định (seed 42). Ảnh tự thu thập (tên tổ `own_`) luôn được đưa vào tập test, để tập test phản ánh điều kiện mà mô hình chưa từng thấy.

Bảng 3: Phân chia dữ liệu phát hiện.

Tập	Số ảnh	Số biển
Train	5 849	[?]
Validation	1 254	1 288
Test	[?]	[?]

3.2 Dữ liệu nhận dạng chữ tổng hợp

Bộ dữ liệu Roboflow chỉ có khung biển số, không có chuỗi ký tự. Để có dữ liệu huấn luyện CRNN, dự án xây dựng bộ sinh biển số tổng hợp (`src/alpr/synth.py`) gồm các bước:

1. **Sinh nội dung** ngẫu nhiên theo đúng định dạng Việt Nam: mã tỉnh 11–99, 50 % biển 2 dòng, phần số 5 chữ số với xác suất 0,8.
2. **Vẽ biển** nền sáng, chữ tối, có viền, bằng font Hershey của OpenCV.
3. **Tăng cường** để mô phỏng ảnh chụp thật: méo phối cảnh, xoay $\pm 5^\circ$, đổi độ sáng và tương phản, làm mờ do lệch nét hoặc do chuyển động, nhiễu cảm biến, giảm độ phân giải.
4. **Tách dòng** biển 2 dòng bằng chính thuật toán dùng lúc suy luận (Mục 4.3).

Tập huấn luyện gồm 50 000 biển (khoảng 75 000 ảnh dòng chữ). Tập kiểm định gồm 2 000 biển sinh với hạt giống khác. Hình 1 minh hoạ một số mẫu.



Hình 1: Biển số tổng hợp sau khi tăng cường. Font Hershey khác font biển số thật, và đây là nguồn gốc của khoảng cách miền phân tích ở Mục 7.

3.3 Định dạng nhãn chữ thật

Để fine-tune CRNN và đánh giá end-to-end, dự án định nghĩa hai tệp nhãn:

- `texts.csv(image, plate_idx, text)`: chữ của từng khung biển, dùng để cắt ảnh dòng chữ huấn luyện. Hai dòng của biển 2 dòng phân cách bằng dấu `/`.
- `test_texts.csv(image, text)`: đáp án cho đánh giá end-to-end.

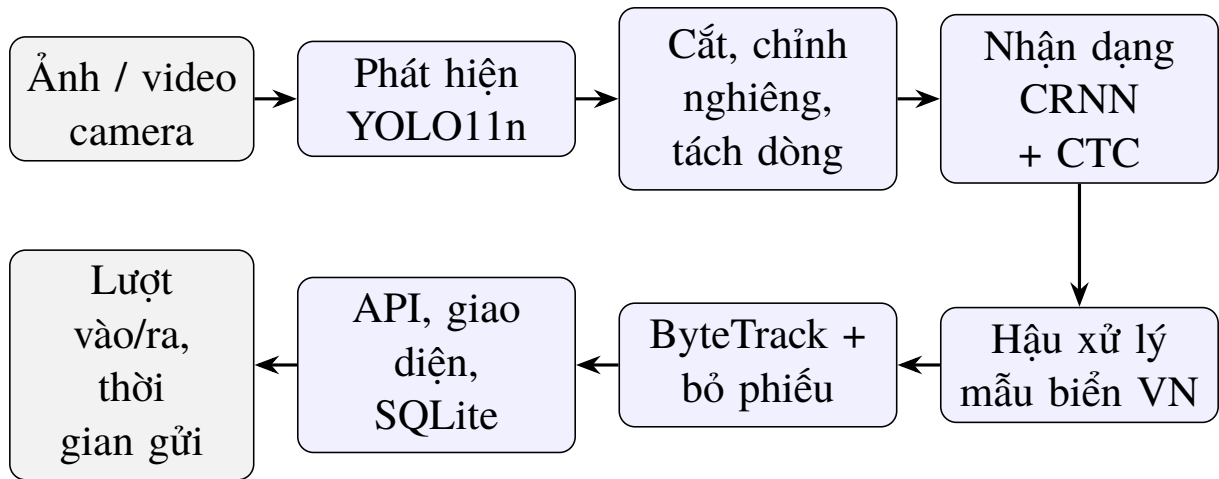
Tại thời điểm viết báo cáo, hai tệp này **chưa được gán nhãn**. Đây là hạn chế chính của kết quả thực nghiệm (Mục 8).

4 Phương pháp

4.1 Kiến trúc tổng thể

Hình 2 mô tả luồng xử lý. Với ảnh tĩnh, mỗi khung biển đi qua các bước từ cắt đến hậu xử lý. Với video, bộ phát hiện chạy ở chế độ theo dõi, và kết quả của từng khung hình được gộp theo track ID.

Mã nguồn tách thành thư viện lõi `src/alpr` (không phụ thuộc web hay CSDL), lớp ứng dụng `app/`, và các script huấn luyện, đánh giá trong `scripts/`. Bộ nhận dạng là một giao diện (protocol) chung: mọi đối tượng có phương thức `recognize(list[img]) -> list[(text, conf)]` đều dùng được. Nhờ đó CRNN và PaddleOCR thay thế được cho nhau, và kiểm thử dùng được bộ nhận dạng giả.



Hình 2: Kiến trúc pipeline VN-ALPR.

4.2 Phát hiện biển số

Bộ phát hiện là YOLO11n khởi tạo từ trọng số tiền huấn luyện trên COCO, rồi fine-tune với một lớp plate. Ngoài các phép tăng cường mặc định của Ultralytics (mosaic, HSV), cấu hình có hai điểm riêng:

- **Tắt lật ngang** (`flip_lr=0`), vì biển số chứa chữ và ảnh lật gương không tồn tại ngoài thực tế.
- Xoay $\pm 10^\circ$ và méo phối cảnh nhẹ để mô phỏng góc đặt camera.

Lúc suy luận, khung có độ tin cậy dưới 0,4 bị loại, và NMS dùng ngưỡng IoU 0,5.

4.3 Tiền xử lý

Cắt và chỉnh nghiêng. Vùng biển được cắt với lề 5% kích thước khung. Góc nghiêng được ước lượng từ hình chữ nhật xoay có diện tích nhỏ nhất (`minAreaRect`) bao vùng chữ đã nhị phân hoá. Góc được tính từ *cạnh dài* của bốn đỉnh hình chữ nhật, thay vì dùng giá trị góc mà OpenCV trả về, vì quy ước của giá trị đó khác nhau giữa các phiên bản OpenCV. Chỉ những góc trong khoảng $[1^\circ, 15^\circ]$ mới được hiệu chỉnh.

Phân loại 1 dòng / 2 dòng. Biển 1 dòng có tỉ lệ rộng/cao khoảng 4,7:1, biển 2 dòng khoảng 1,4:1. Ảnh cắt được coi là biển 2 dòng khi $w/h < 2,5$.

Tách dòng bằng hồ sơ chiếu ngang. Gọi $B(x, y) \in \{0, 1\}$ là ảnh nhị phân, trong đó điểm chữ có giá trị 1, và h là chiều cao ảnh. Hồ sơ chiếu ngang là

$$P(y) = \sum_x B(x, y). \quad (1)$$

Sau khi làm trơn P bằng bộ lọc trung bình 3 điểm, dòng tách y^* là tâm của đoạn các hàng có giá trị nhỏ nhất trong dải giữa $[0, 3h, 0, 7h]$, tức là hàng “trống” nhất giữa hai dòng chữ. Giới hạn trong dải giữa giúp tránh nhầm với viền trên và viền dưới của biển.

4.4 Nhận dạng ký tự: CRNN

Ảnh mỗi dòng được chuyển sang xám, co giãn giữ nguyên tỉ lệ về chiều cao 32, đệm bên phải tới chiều rộng 128, rồi chuẩn hoá về $[-1, 1]$. Kiến trúc mạng được trình bày ở Bảng 4.

Bảng 4: Kiến trúc CRNN (tổng cộng 2,35 triệu tham số). Mỗi khối Conv gồm Conv 3×3 , BatchNorm và ReLU.

Lớp	Cấu hình	Kích thước đầu ra ($C \times H \times W$)
Đầu vào	ảnh xám	$1 \times 32 \times 128$
Conv + MaxPool	64, pool 2×2	$64 \times 16 \times 64$
Conv + MaxPool	128, pool 2×2	$128 \times 8 \times 32$
Conv $\times 2$ + MaxPool	256, pool 2×1	$256 \times 4 \times 32$
Conv + MaxPool	256, pool 2×1	$256 \times 2 \times 32$
Dropout2d	$p = 0,1$	$256 \times 2 \times 32$
Trung bình theo chiều cao		32×256 (chuỗi $T = 32$ bước)
BiLSTM	2 lớp, 128 đơn vị mỗi chiều	32×256
Tuyến tính		32×37 (36 ký tự + blank)

Hai lựa chọn thiết kế nhằm giữ mô hình xuất được sang ONNX:

- Lấy trung bình theo chiều cao bằng `mean(dim=2)` thay vì `AdaptiveAvgPool2d((1, None))`, vì phép sau không xuất được sang ONNX.
- Xuất ONNX bằng trình xuất cũ (`dynamo=False`), vì trình xuất dynamo cố định kích thước batch của LSTM vào đồ thị tính toán.

Hàm mất mát CTC. Với bộ ký tự $\mathcal{A}$ gồm 36 ký tự (0–9, A–Z) và ký hiệu trống (blank, chỉ số 0), mạng cho ra phân phối $p(\cdot | x)$ tại mỗi bước $t = 1, \dots, T$. Gọi $\mathcal{B}$ là ánh xạ gộp các ký tự lặp liên tiếp rồi bỏ blank. Xác suất của chuỗi đáp án y là tổng xác suất mọi đường đi π ánh xạ về y :

$$p(y | x) = \sum_{\pi \in \mathcal{B}^{-1}(y)} \prod_{t=1}^T p(\pi_t | x), \quad \mathcal{L}_{\text{CTC}} = -\log p(y | x). \quad (2)$$

Lúc suy luận, chuỗi được giải mã tham lam (greedy): lấy ký hiệu có xác suất lớn nhất ở mỗi bước rồi áp dụng $\mathcal{B}$. Độ tin cậy là trung bình xác suất của các ký tự được giữ lại.

4.5 Hậu xử lý theo định dạng biển số Việt Nam

Chuỗi OCR trước hết được chuẩn hoá: chuyển sang chữ hoa, đổi Đ→D, chỉ giữ A–Z và 0–9. Sau đó thuật toán thử từng mẫu đầu biển m ở Bảng 2, ghép với phần số 4–5 chữ số:

Listing 1: Thuật toán khớp mẫu biển số (rút gọn từ `plate_format.py`).

```
best = None
for m in TEMPLATES:
    # thứ tự mẫu dùng để phá thế hoà
    for n in (5, 4):
        # độ dài phần số
        if len(s) != len(m) + n: continue
        ok, text, fixes = True, "", 0
```

```

for ch, kind in zip(s, m + "D" * n):
    if kind == "D" and not ch.isdigit():
        if ch not in TO_DIGIT: ok = False; break
        ch = TO_DIGIT[ch]; fixes += 1      # 0->0, B->8, S->5, ...
    if kind == "L" and ch not in VALID_LETTERS:
        if ch not in TO_LETTER: ok = False; break
        ch = TO_LETTER[ch]; fixes += 1     # 0->D, 8->B, 5->S, ...
    text += ch
if ok and (best is None or fixes < best.fixes):
    best = (text, fixes)

```

Ba quy tắc bổ sung:

- **Ràng buộc từ biển 2 dòng.** Độ dài dòng 1 cho biết seri kết thúc ở đâu. Tuy vậy, nếu một phép khớp trên chuỗi đã ghép cần ít lần sửa hơn, phép khớp đó được ưu tiên, để bù cho trường hợp tách dòng lệch một ký tự.
- **Thứ tự ưu tiên khi hoà.** Biển 1 dòng ưu tiên mẫu ô tô (DDL, DDLL). Biển 2 dòng ưu tiên mẫu xe máy (DDL D).
- **Mẫu DDLLD** chỉ chấp nhận seri MD, tức biển xe máy điện MĐ.

Chế độ `fix=False` chỉ chấp nhận chuỗi khớp đúng mẫu mà không sửa ký tự nào. Chế độ này dùng cho thí nghiệm ablation.

4.6 Theo dõi và bỏ phiếu qua nhiều khung hình

Trong video, mỗi xe xuất hiện ở nhiều khung hình và mỗi lần đọc có thể khác nhau. Với mỗi track ID, bộ bỏ phiếu cộng dồn điểm có trọng số cho từng chuỗi hợp lệ s :

$$S(s) = \sum_{i: \hat{s}_i=s} \max(c_i, 10^{-3}), \quad (3)$$

trong đó c_i là độ tin cậy OCR của lần đọc thứ i . Gọi $s^* = \arg \max_s S(s)$. Track được *xác nhận* và phát ra đúng một sự kiện khi thoả cả hai điều kiện sau:

$$\text{số lần đọc hợp lệ} \geq 3 \quad \text{và} \quad \frac{S(s^*)}{\sum_s S(s)} \geq 0,6. \quad (4)$$

Sau khi được xác nhận, track không cần chạy OCR nữa, nhờ đó tiết kiệm tính toán. Track mất dấu quá 30 khung hình bị loại. Nếu track đó chưa được xác nhận nhưng đã có đủ số lần đọc, kết quả tốt nhất của nó vẫn được phát ra.

4.7 Lớp ứng dụng

- **CSDL vào/ra (SQLite).** Một lượt cùng biển và cùng hướng trong vòng 60 giây bị bỏ qua, vì đó là cùng một xe được đọc hai lần. Khi xe ra mà trước đó có lượt vào, hệ thống tính thời gian gửi `duration_s`.
- **REST API (FastAPI)** gồm các điểm cuối `POST /recognize`, `POST /gate/{in|out}`, `GET /events` và `GET /parked`. Pipeline được khởi tạo trễ (lazy), nên kiểm thử có thể đưa vào pipeline giả.
- **Giao diện Streamlit:** tải ảnh hoặc video lên, xem kết quả trực quan và tra cứu lịch sử.

5 Thiết lập thực nghiệm

5.1 Môi trường

Mô hình được huấn luyện trên Google Colab với GPU NVIDIA T4 (15 GB) bằng notebook `notebooks/train_colab.ipynb`. Tốc độ suy luận được đo trên laptop với CPU Intel Core i5-10300H (4 nhân, 8 luồng) và 14 GB RAM, không dùng GPU.

5.2 Siêu tham số

Bảng 5: Siêu tham số huấn luyện.

Bộ phát hiện (YOLO11n)	
Khởi tạo	<code>yolo11n.pt</code> (tiền huấn luyện trên COCO)
Kích thước ảnh, batch	640, 32
Số epoch	tối đa 100, dừng sớm sau 20 epoch không cải thiện
Bộ tối ưu	AdamW, $\text{lr} = 0,002$, momentum 0,9 (Ultralytics tự chọn)
Tăng cường	mosaic (tắt ở 10 epoch cuối), xoay $\pm 10^\circ$, phối cảnh $5 \cdot 10^{-4}$, HSV-V 0,5, không lật ngang
Bộ nhận dạng (CRNN)	
Dữ liệu	50 000 biển tổng hợp (khoảng 75 000 dòng); kiểm định 2 000 biển
Batch, epoch	128, 20 (checkpoint tốt nhất ở epoch 7)
Bộ tối ưu	AdamW, $\text{lr} = 10^{-3}$, weight decay 10^{-4} , lịch OneCycle
Khác	cắt gradient ở chuẩn 5, CTC blank = 0, tăng cường trực tuyến với cường độ 0,7

5.3 Độ đo

- **Phát hiện:** precision, recall, $\text{mAP}@0.5$ và $\text{mAP}@0.5:0.95$ (trung bình trên các ngưỡng IoU từ 0,5 đến 0,95).
- **Độ chính xác biển** (khớp hoàn toàn): tỉ lệ biển có chuỗi dự đoán trùng hoàn toàn với đáp án sau khi chuẩn hoá.
- **Độ chính xác ký tự:** $1 - \text{CER}$, với

$$\text{CER} = \frac{\sum_i \text{lev}(\hat{y}_i, y_i)}{\sum_i |y_i|}, \quad (5)$$

trong đó lev là khoảng cách Levenshtein.

- **End-to-end:** các biển đáp án trong một ảnh được ghép tham lam với các dự đoán theo khoảng cách chỉnh sửa. Biển bị bỏ sót được tính là sai.

6 Kết quả

6.1 Bộ phát hiện

Ngay sau epoch đầu tiên, trên tập kiểm định (1 254 ảnh, 1 288 biển), mô hình đã đạt **mAP@0.5 = 0,969** và $\text{mAP@0.5:0.95} = 0,552$. Như vậy chỉ tiêu mAP@0.5 đã đạt từ rất sớm, và khó khăn chủ yếu nằm ở độ khít của khung (mAP@0.5:0.95).

Bảng 6: Kết quả bộ phát hiện.

Thời điểm	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Epoch 1, tập val	0,976	0,927	0,969	0,552
Mô hình cuối, tập test	[?]	[?]	[?]	[?]
Mục tiêu			$\geq 0,95$	$\geq 0,70$

6.2 Bộ nhận dạng trên dữ liệu tổng hợp

CRNN sau giai đoạn huấn luyện trước (bản ONNX) được đánh giá trên 1 000 biển tổng hợp *mới*, sinh với hạt giống chưa dùng khi huấn luyện:

Bảng 7: CRNN trên 1 000 biển tổng hợp chưa từng thấy (1 494 dòng).

Chỉ số	Giá trị
Độ chính xác dòng	85,4 %
Độ chính xác biển, OCR thô	78,8 %
Độ chính xác biển, có hậu xử lý	78,8 %
Biển 1 dòng (506 biển)	76,7 %
Biển 2 dòng (494 biển)	81,0 %

Hậu xử lý không cải thiện kết quả trên dữ liệu tổng hợp. Điều này hợp lý: các lỗi trên ảnh tổng hợp chủ yếu là thiếu hoặc thừa ký tự khi ảnh bị làm mờ mạnh, chứ không phải các cặp ký tự dễ nhầm như 0/0 hay B/8 mà hậu xử lý được thiết kế để sửa. Tác dụng của hậu xử lý cần được đo trên dữ liệu thật (Bảng 8).

6.3 Ablation end-to-end

Bảng 8 là khung thí nghiệm ablation được tạo bởi `scripts/evaluate.py`. Các số liệu sẽ được điền sau khi có `test_texts.csv`.

Bảng 8: Ablation end-to-end trên tập test.

Cấu hình	Acc. biển	Acc. 1 dòng	Acc. 2 dòng	Acc. ký tự
YOLO + PaddleOCR	[?]	[?]	[?]	[?]
YOLO + PaddleOCR + hậu xử lý	[?]	[?]	[?]	[?]
YOLO + CRNN	[?]	[?]	[?]	[?]
YOLO + CRNN + hậu xử lý	[?]	[?]	[?]	[?]

6.4 Tốc độ

Tốc độ end-to-end (phát hiện, tiền xử lý, OCR 2 dòng và hậu xử lý) được đo trên ảnh 872×541 chứa một biển 2 dòng. Mỗi cấu hình chạy 30 lần sau 3 lần khởi động.

Bảng 9: Độ trễ trên CPU Intel i5-10300H.

Cấu hình	Trung vị (ms)	P90 (ms)	FPS (theo trung vị)
PyTorch (.pt)	61,9	78,2	16,2
ONNX Runtime (.onnx)	140,5	170,4	7,1
Mục tiêu (CPU)			≥ 8

Bản PyTorch đạt khoảng 16 FPS trên CPU, vượt mục tiêu 8 FPS. Kết quả bản ONNX chậm hơn là điều đáng chú ý. Nhiều khả năng nguyên nhân là mô hình ONNX được xuất với đầu vào cố định 640×640 , trong khi Ultralytics với .pt có ảnh chữ nhật về 640×416 , nên phải xử lý ít điểm ảnh hơn khoảng 35 %. Một hướng cần thử tiếp là xuất ONNX với kích thước động (dynamic=True) hoặc với imgsz chữ nhật.

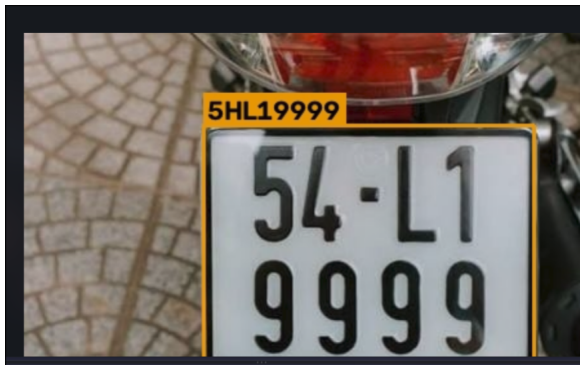
7 Phân tích lỗi

7.1 Nghiên cứu tình huống: biển 54-L1 9999

Hình 3 là một ảnh thật mà hệ thống đọc sai thành 5HL19999. Kiểm tra từng bước cho thấy:

Bảng 10: Lăn theo lỗi qua từng bước của pipeline.

Bước	Kết quả	Đánh giá
Phát hiện	khung bao trọn biển, độ tin cậy 0,85	đúng
Phân loại số dòng	2 dòng	đúng
Tách dòng	54-L1 và 9999, ảnh sạch	đúng
CRNN, dòng 1	5HL1 (đáp án 54L1)	sai: 4→H
Hậu xử lý	không khớp mẫu nào nên biển bị đánh dấu không hợp lệ	không cứu được



(a) Kết quả trên giao diện.

(b) Hai ảnh dòng chữ CRNN nhận được (32×128 , phóng to).**Hình 3:** Tình huống lỗi 54-L1 9999 $\rightarrow$ 5HL19999.

Nguyên nhân. Lỗi nằm hoàn toàn ở CRNN, và gốc rễ là **khoảng cách miền** giữa dữ liệu tổng hợp và dữ liệu thật. Trong font Hershey dùng để sinh dữ liệu, số 4 có đỉnh khép kín. Trong font biển số Việt Nam, số 4 hở đỉnh, gồm hai nét dọc nối bằng một gạch ngang, nên hình dạng rất gần chữ H. Mô hình chưa từng thấy kiểu 4 này nên chọn chữ gần nhất trong những gì đã học.

Hậu xử lý cũng không sửa được, vì bảng TO_DIGIT chỉ chứa các cặp nhầm phổ biến trong font Latin ($0 \rightarrow 0$, $B \rightarrow 8$, ...), không có cặp $H \rightarrow 4$, một cặp chỉ xuất hiện với font biển Việt Nam.

7.2 Nhận xét chung

- Bộ phát hiện và khối tiền xử lý hoạt động tốt trên ảnh thật. Điểm nghẽn của hệ thống là bộ nhận dạng.
- Độ chính xác trên dữ liệu tổng hợp (Bảng 7) *không* phản ánh độ chính xác trên biển thật. Đánh giá end-to-end trên dữ liệu thật là bắt buộc.
- Thiết kế hậu xử lý theo mẫu vẫn hợp lý, nhưng bảng nhầm lẫn nên được xây dựng từ lỗi thực tế (tệp outputs/errors_*.csv) thay vì chỉ dựa trên trực giác.

8 Hạn chế và hướng phát triển

Hạn chế.

1. Chưa có nhãn chữ thật, nên CRNN chưa được fine-tune trên biển thật và chưa có số liệu ablation end-to-end.
2. Dữ liệu tổng hợp dùng font Hershey, khác font biển số Việt Nam.
3. Chưa tối ưu cho ảnh ban đêm hay camera hồng ngoại.
4. Hướng vào/ra hiện cố định theo camera.

Hướng phát triển, theo thứ tự ưu tiên.

1. **Gán nhãn chữ bán tự động.** PaddleOCR đọc trước trên các khung biển có sẵn trong nhãn YOLO, sau đó người kiểm tra và sửa. Từ đó tạo `texts.csv` và `test_texts.csv`, rồi fine-tune CRNN (bước 4b) và chạy ablation.
2. Sinh dữ liệu tổng hợp bằng font gần giống font biển số Việt Nam, để giảm khoảng cách miền ngay từ giai đoạn huấn luyện trước.

3. Bổ sung các cặp nhằm đặc thù của font Việt Nam (ví dụ H→4) vào hậu xử lý, dựa trên phân tích lỗi thực tế.
4. Xuất ONNX với kích thước đầu vào động để tăng tốc trên CPU. Thử TensorRT FP16 khi triển khai trên GPU.
5. Suy ra hướng vào/ra từ quỹ đạo theo dõi thay vì cố định theo camera.

9 Kết luận

Dự án đã xây dựng một hệ thống ALPR hoàn chỉnh cho biển số Việt Nam, xử lý được cả biển 1 dòng và 2 dòng, đi từ phát hiện, nhận dạng, hậu xử lý theo định dạng biển, theo dõi qua video đến ứng dụng ghi nhận vào/ra. Bộ phát hiện YOLO11n đạt mục tiêu mAP@0.5 ngay sau epoch đầu tiên, và pipeline chạy khoảng 16 FPS trên CPU laptop. Phân tích lỗi chỉ ra rằng khoảng cách giữa font tổng hợp và font biển thật là điểm nghẽn chính của bộ nhận dạng. Bước tiếp theo có tác động lớn nhất là gán nhãn chữ cho ảnh thật để fine-tune CRNN và hoàn thiện đánh giá end-to-end.

Tài liệu

- [1] J. Redmon, S. Divvala, R. Girshick, A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection. In *CVPR*, 2016.
- [2] G. Jocher, J. Qiu. Ultralytics YOLO11. <https://github.com/ultralytics/ultralytics>, 2024.
- [3] B. Shi, X. Bai, C. Yao. An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition. *IEEE TPAMI*, 39(11):2298–2304, 2017.
- [4] A. Graves, S. Fernández, F. Gomez, J. Schmidhuber. Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. In *ICML*, 2006.
- [5] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, X. Wang. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. In *ECCV*, 2022.
- [6] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, J. Zhou, Y. Bai, Z. Yu, Y. Yang, Q. Dang, H. Wang. PP-OCR: A Practical Ultra Lightweight OCR System. *arXiv:2009.09941*, 2020.

A Cấu trúc mã nguồn

src/alpr/	thư viện lõi
detection/	YOLO: detect() cho ảnh, track() cho video (ByteTrack)
preprocess.py	cắt, chỉnh nghiêng, phân loại và tách dòng
ocr/crnn/	mô hình, bộ ký tự, dataset, bộ nhận dạng .pt/.onnx
ocr/paddle.py	baseline PaddleOCR
postprocess/	khớp mẫu biển số Việt Nam
tracking/	bỏ phiếu theo track ID

pipeline.py	ghép các khối
app/	FastAPI, SQLite, Streamlit
scripts/	dữ liệu, huấn luyện, đánh giá, xuất, benchmark
tests/	pytest -- dùng mô hình giả nên không cần trọng số

B Tái lập kết quả

```
# huấn luyện: notebooks/train_colab.ipynb trên Colab (GPU T4)
pip install -r requirements.txt && pip install -e .
pytest -q                                     # kiểm thử
streamlit run app/ui/streamlit_app.py         # giao diện web
uvicorn app.api.main:app --port 8000          # REST API
python scripts/evaluate.py --images data/processed/detection/images/test \
    --gt data/raw/test_texts.csv --tag "YOLO + CRNN + postprocess"
```